

G-Code Audit Engine — Test Suite Summary

Repo: `gcode-audit` **Version:** v5 (test infrastructure) **Date:** 2026-05-05 **Total tests:** 80, all passing **Run time:** ~0.06s

This document lists every test in the suite, grouped by file and topic. Each test has a brief plain-English description of the behavior it locks in.

How the suite is organized

File	Tests	Purpose
<code>test_parser.py</code>	41	Parser layer: tokenization, comments, modal state
<code>test_g91.py</code>	9	G91 incremental coordinate handling (added in v4)
<code>test_validators.py</code>	21	All four validators with positive AND negative cases
<code>test_regression.py</code>	9	Locks in expected output for <code>tests/gcode/test.gcode</code>
Total	80	

`test_parser.py` — 41 tests

TestTokenizer (11 tests)

The tokenizer is the first thing G-code touches. These tests lock in that it handles every common form correctly.

Test	What it verifies
<code>test_space_separated_tokens</code>	Standard space-separated form <code>G1 X10 Y20</code> tokenizes correctly
<code>test_no_space_tokens</code>	No-space form <code>G1X10Y20Z-1.5</code> splits into separate tokens
<code>test_zero_padded_g_code_normalized</code>	<code>G01</code> is normalized to <code>G1</code> (Fanuc/Haas post-processors emit zero-padded codes)
<code>test_zero_padded_m_code_normalized</code>	<code>M03</code> is normalized to <code>M3</code> (same reason)
<code>test_g00_normalized_to_g0</code>	<code>G00</code> is normalized to <code>G0</code>
<code>test_negative_coordinates_preserved</code>	<code>X-1.5 Y-2.0 Z-0.001</code> preserves signs correctly
<code>test_decimal_coordinates_preserved</code>	Decimal precision is preserved through tokenization
<code>test_mixed_case_normalized_to_upper</code>	<code>g1 x10 y20</code> is normalized to uppercase
<code>test_arc_parameters</code>	Arc parameters <code>I</code> , <code>J</code> are preserved as tokens
<code>test_empty_string_yields_no_tokens</code>	Empty input produces empty token list
<code>test_whitespace_only_yields_no_tokens</code>	Whitespace-only input produces empty token list

TestCommentStripping (6 tests)

G-code allows two comment styles: `(...)` and `;...`. Both must be stripped before tokenization.

Test	What it verifies
<code>test_parenthetical_comment_at_end_of_line</code>	<code>G1 X10 (rapid here)</code> removes comment, keeps code
<code>test_parenthetical_comment_mid_line</code>	<code>G1 X10 (note) Y20</code> keeps both code segments
<code>test_multiple_parenthetical_comments</code>	Multiple <code>(...)</code> comments on one line all stripped
<code>test_semicolon_comment_to_end_of_line</code>	<code>G1 X10 ; comment</code> keeps code, drops comment
<code>test_semicolon_overrides_parens_after_it</code>	Anything after <code>;</code> is dropped, even unmatched parens
<code>test_entire_line_comment</code>	<code>(entire line is a comment)</code> produces empty result

TestLineClassification (6 tests)

Every parsed line is tagged as `CODE`, `COMMENT`, or `EMPTY`.

Test	What it verifies
<code>test_blank_line_classified_empty</code>	<code>" "</code> is classified as <code>EMPTY</code>
<code>test_whitespace_only_classified_empty</code>	Whitespace-only lines are classified as <code>EMPTY</code>

test_pure_comment_classified_comment	(comment) line is classified as COMMENT
test_semicolon_only_line_classified_comment	; comment line is classified as COMMENT
test_code_line_classified_code	Line with G-code is classified as CODE
test_code_with_inline_comment_classified_code	G1 X10 (note) is classified as CODE

TestStatePersistence (5 tests)

Modal state must carry forward across lines until something changes it.

Test	What it verifies
test_position_persists_through_blank_lines	X/Y/Z values survive blank lines
test_motion_mode_inherited_on_continuation_line	Motion mode (G0/G1) inherited when no G-code present
test_feed_rate_persists_until_changed	Feed rate (F) carries forward and is updated when reset
test_spindle_state_tracked	M3/M5 transitions update spindle state correctly
test_tool_number_tracked	T-number is captured into state

TestModalCodes (5 tests)

The various G-code modal groups each map to their correct state slot.

Test	What it verifies
test_g90_captured_as_distance	G90 sets state["distance"]
test_g21_captured_as_units	G21 sets state["units"]
test_g17_captured_as_plane	G17 sets state["plane"]
test_motion_modes_captured	G0, G1, G2, G3 all set state["motion"]
test_unknown_g_code_does_not_set_motion	G54 (work coordinate) does not pollute motion state

TestParserOutputShape (5 tests)

The parser output format is part of the public contract. These tests make sure it stays stable.

Test	What it verifies
test_returns_dict_with_lines_key	Output is a dict with a lines key
test_returns_parser_issues_key	Output has parser_issues key (always present, possibly empty)
test_each_line_entry_has_required_keys	Each entry has line_index, state, tokens, line_type
test_line_indices_are_one_based	Line indices start at 1, not 0
test_state_is_independent_per_line	Each entry’s state is a snapshot, not a shared reference

TestInputValidation (3 tests)

Defensive input handling — the parser should reject obviously bad inputs cleanly rather than crashing mysteriously.

Test	What it verifies
test_non_list_input_raises_value_error	Passing a string instead of a list raises ValueError
test_non_string_line_raises_value_error	Non-string entries in the list raise ValueError
test_empty_input_returns_empty_lines	Empty list returns empty lines and parser_issues

test_g91.py

— 9 tests

Tests for G91 incremental coordinate interpretation, added in v4.

Baseline behavior (1 test)

Test	What it verifies
test_g90_absolute_mode_replaces_position	G90 mode: each X/Y/Z replaces the previous value (unchanged from v3)

Core G91 incremental behavior (3 tests)

Test	What it verifies
test_g91_increments_from_prior_position	G91 mode: each X/Y/Z is added to the previous value
test_g91_to_g90_round_trip	G90 → G91 → G90: each switch correctly changes interpretation
test_g91_persists_across_continuation_lines	G91 mode and motion mode both persist on continuation lines

Edge cases (2 tests)

Test	What it verifies
test_g91_no_prior_position_emits_info_issue	G91 active at startup: first axis use treated as absolute, INFO issue emitted
test_g91_no_prior_position_only_first_axis_use_flagged	Once an axis has any prior value, subsequent G91 moves don't re-flag

Mid-line ordering (3 tests)

These verify the two-pass token application: G-codes apply *before* coordinate words on the same line.

Test	What it verifies
test_g91_takes_effect_before_coordinate_on_same_line	G91 X5 interprets X5 as incremental
test_coordinate_order_does_not_matter_within_a_line	X3 G91 behaves identically to G91 X3
test_g90_takes_effect_before_coordinate_on_same_line	G90 X100 interprets X100 as absolute

test_validators.py — 21 tests

For each validator, both **positive cases** (known-bad programs that SHOULD trigger the issue) and **negative cases** (known-good programs that SHOULD NOT trigger it). Negative cases catch “validator now flags everything” regressions.

TestStartupSequence (4 tests)

Test	Sense	What it verifies
test_clean_startup_no_issues	Negative	Lift Z, move XY, plunge — no issues
test_cut_before_safe_z_flagged_critical	Positive	Plunge with no prior Z lift triggers CRITICAL
test_xy_motion_before_safe_z_flagged_warning	Positive	XY motion before Z lift triggers WARNING
test_safe_z_then_xy_then_cut_no_issues	Negative	Proper Z-then-XY-then-cut sequence is clean

TestGroupOperations (4 tests)

Test	What it verifies
test_no_cutting_means_no_operations	Programs with Z always above 0 produce no operations
test_single_cutting_pass	Contiguous run of Z<=0 lines forms one operation
test_two_separated_cutting_passes	Two cuts separated by Z lift form two operations
test_operation_open_at_eof_still_recorded	Cut still in progress at end of file is closed and recorded

TestProfileDepths (5 tests)

Test	What it verifies
test_empty_operations_yields_empty_profile	No operations → empty profile
test_single_operation_yields_single_cluster	One operation → one-cluster profile
test_similar_depths_cluster_within_tolerance	Depths within 0.01 are merged into one cluster
test_distinct_depths_form_separate_clusters	Genuinely different depths produce separate clusters
test_clusters_sorted_by_depth_ascending	Output sorted from deepest to shallowest

TestSpindleAndFeed (8 tests)

Test	Sense	What it verifies
test_clean_program_no_issues	Negative	Spindle on, feed set, then cut — no issues
test_cut_with_spindle_off_flagged_critical	Positive	G1 after M5 triggers CRITICAL
test_cut_with_spindle_never_started_flagged_warning	Positive	G1 with no prior M3/M4 triggers WARNING
test_cut_without_feed_rate_flagged_warning	Positive	G1 with no prior F triggers WARNING
test_zero_padded_m3_normalized_correctly	Negative	M03 (zero-padded) is treated as M3, no false positive
test_zero_padded_m5_normalized_correctly	Positive	M05 (zero-padded) triggers spindle-off detection
test_g0_rapids_do_not_trigger_spindle_check	Negative	G0 rapids without spindle/feed are not flagged
test_issue_reported_once_per_violation_window	Negative	Repeated violations in the same window emit one issue, not many

test_regression.py

— 9 tests

Locks in the current expected behavior on the real-world `test.gcode` file (Autodesk Fusion / OpenBuilds GRBL post-processor output, 110 lines, 2 cutting operations). Drift here forces a conscious review.

TestRegressionParser (4 tests)

Test	Locked-in value
test_total_line_count	110 lines parsed
test_line_type_breakdown	64 CODE / 39 COMMENT / 7 EMPTY
test_no_parser_issues	Empty parser_issues list
test_final_modal_state	X=-10.0, Y=-10.0, Z=25.4, motion=G0, distance=G90, units=G21, plane=G17, spindle=M5, feed=2800.0, speed=18000.0

TestRegressionValidators (5 tests)

Test	Locked-in value
test_no_startup_sequence_issues	Empty list (program is clean)
test_two_cutting_operations	Exactly 2 operations grouped
test_depth_profile_two_clusters	Exactly 2 depth clusters
test_depth_profile_values	Depths -9.525 and -3.31, count of 1 each
test_no_spindle_or_feed_issues	Empty list (program is clean)

Coverage gaps (for future expansion)

- Areas not currently covered by tests, organized by priority:
- High priority (next pass):** - Comprehensive G92 (set-position) handling — currently not tracked - Arc geometry validation — currently no tests because feature unimplemented
 - Medium priority:** - More real-world G-code files in `tests/gcode/` for regression coverage across different post-processors (Fanuc, Haas, LinuxCNC, etc.) - Performance/scale tests on large files (10K+ line programs)
 - Low priority:** - Property-based tests (e.g. with Hypothesis) for the tokenizer

This document was generated alongside v5. It will be updated as new tests are added.